bancaRing - Transazioni bancarie distribuite con mutua esclusione a Token Ring

Progetto di Intelligenza Artificiale Distribuita. Quattro ATM (processi separati su `localhost`) modificano un unico saldo bancario condiviso. La mutua esclusione è garantita **solo** dall'algoritmo Token Ring; in più il sistema tollera il crash di un nodo e la perdita del token (rigenerazione).

- **Repository GitHub:** DA_INSERIRE
- **Video dimostrativo:** DA_INSERIRE

1. Istruzioni di esecuzione

Elemento	Valore
Linguaggio	Java 25
Build	Maven 3.9+
Librerie / dipendenze	nessuna (solo JDK: `java.net`, `java.util.concurrent`)
Porte usate	`5001`-`5004` su localhost (ATMn ascolta su `5000+n`)

Compilazione (dalla cartella del progetto, con JDK 25 attivo):

```
mvn package
```

Produce `target/bancaRing.jar`.

Avvio dei 4 nodi - un terminale per nodo, ognuno nella cartella del progetto. Il numero passato al programma è l'id del nodo: ogni terminale = un processo JVM = un ATM, con memoria privata propria. Avviare **ATM1 per ultimo** (crea il token e deve trovare gli altri già in ascolto).

Terminale	Comando
4	`java -jar target/bancaRing.jar 4 w500`
3	`java -jar target/bancaRing.jar 3 d100`
2	`java -jar target/bancaRing.jar 2 w200`
1 (ultimo)	`java -jar target/bancaRing.jar 1`

Gli argomenti dopo l'id sono transazioni da mettere in coda: `dN` = deposito di N, `wN` = prelievo di N. Si possono anche digitare a runtime nel terminale del nodo (es. `d100`, `w 50`). Il comando `crash` fa morire il nodo non appena riceve il token (serve alla demo del guasto).

Avvio automatico (Windows): `.\demo.ps1` apre 4 finestre e riproduce l'esempio del testo; `.\demo.ps1 -Scenario crash` aggiunge il guasto di ATM3.

Riavvio di un nodo morto: `java -jar target/bancaRing.jar <id> --rejoin` (senza `--rejoin` ATM1 creerebbe un secondo token).

2. Architettura

Anello logico `ATM1 -> ATM2 -> ATM3 -> ATM4 -> ATM1`. In funzionamento normale ogni nodo parla solo con il proprio successore.

- **Nessuna memoria condivisa: il saldo viaggia dentro il token.** Il messaggio `TOKEN;epoch;seq;saldo` è l'unico posto in cui il saldo esiste "ufficialmente". Chi ha il token può leggerlo e modificarlo; gli altri non possono. La mutua esclusione è quindi garantita per costruzione.
- **Messaggi** (TCP, una riga di testo, richiesta/risposta): `TOKEN;epoch;seq;saldo -> ACK`; `QUERY -> STATE;vivo;epoch;seq;saldo` (usato solo nel recupero).
- **Sezione critica (una transazione per visita del token):** lettura del saldo, validazione (un prelievo con saldo insufficiente viene rifiutato), aggiornamento, scrittura nel token, log. La transazione si considera confermata quando il token con il nuovo saldo viene inoltrato.
- Se un nodo non ha transazioni inoltra il token subito (dopo una breve pausa di 0,8 s solo per rendere leggibile il log).

3. Tolleranza ai guasti (modello crash-stop)

Problema	Soluzione
Il successore è morto	Il token è considerato consegnato solo dopo l'`ACK`; senza ACK si prova il nodo successivo (l'anello si riconfigura saltando i morti).
Il token si perde (il nodo che lo possedeva muore)	Ogni nodo conserva l'ultimo stato visto (epoch, seq, saldo). Un watchdog scatta se il token non si vede per `8 s + 2 s * id` (il nodo con id minore recupera per primo).
Falso allarme (token lento, non perso)	Prima di rigenerare il nodo interroga gli altri con `QUERY`: se uno ha visto il token da poco, non fa nulla.
Rigenerazione	Si prende la copia più recente tra tutti i nodi raggiungibili e si crea un token con `epoch + 1`.
Due token (duplicato)	Un token con `(epoch, seq)` non più recente di quello già visto viene scartato.

Limiti dichiarati: non si gestiscono partizioni di rete; per riconfigurare l'anello ogni nodo conosce le porte di tutti (ma usa solo il successore finché è vivo); una transazione eseguita da un nodo che muore prima di inoltrare il token non è confermata e viene persa insieme alla sua coda, il saldo resta coerente.

4. Esempi di log

Scenario base (saldo iniziale 1000; ATM2 w200, ATM3 d100, ATM4 w500):

```
[ATM1] TOKEN RICEVUTO (epoch=1 seq=0 saldo=1000)
[ATM1] TOKEN INOLTRATO a ATM2
[ATM2] TOKEN RICEVUTO (epoch=1 seq=1 saldo=1000)
```

```
[ATM2] INIZIO transazione: PRELIEVO 200
[ATM2] FINE transazione: PRELIEVO 200 -> saldo aggiornato = 800
[ATM2] TOKEN INOLTRATO a ATM3
[ATM3] TOKEN RICEVUTO (epoch=1 seq=2 saldo=800)
[ATM3] INIZIO transazione: DEPOSITO 100
[ATM3] FINE transazione: DEPOSITO 100 -> saldo aggiornato = 900
...
[ATM4] FINE transazione: PRELIEVO 500 -> saldo aggiornato = 400
```

Scenario crash (ATM3 muore con il token, estratto di una esecuzione reale):

```
[ATM3] *** CRASH simulato: il token e' perso con me ***
[ATM1] TIMEOUT: token sospetto perso, interrogo gli altri nodi
[ATM1] ATM3 non risponde (morto)
[ATM1] TOKEN RIGENERATO: epoch 2, saldo recuperato = 850
[ATM2] ATM3 non raggiungibile
[ATM2] TOKEN INOLTRATO a ATM4 (saltati 1 nodi morti)
[ATM4] FINE transazione: PRELIEVO 500 -> saldo aggiornato = 350
```

Il saldo 850 è quello confermato prima che ATM3 ricevesse il token; il deposito di ATM3, mai inoltrato, non è confermato.

5. Contenuto del repository

- `src/main/java/org/ia/distribuita/BancaRing.java` - tutto il codice (token, messaggi, sezione critica, recupero), commentato
- `pom.xml` - build Maven (Java 25)
- `demo.ps1` - avvio automatico dei 4 terminali